

Certified-Depth Accumulation: a frozen weak model matches a strong one by compressing and banking verified knowledge

Working draft v0 — progress report. All numbers are from our own runs and are honest, including the negative ones. This is a mechanism paper, not a "we beat SOTA everywhere" paper.

Abstract

A weak language model (gpt-4o-mini) does not get smarter per forward pass — its weights are frozen. We ask a different question: can it reach the *effective* capability of a strong model (gpt-4o) on tasks that admit an executable check, without any fine-tuning?

We give one task-agnostic mechanism, **Compression-as-Cognition with Certified-Depth Accumulation**. Solving a task is finding the shortest program that *reproduces the observations under execution*; bits, not a model, are the judge. On top of this we add (i) **Variation-MDL**, which extracts a reliable answer from the divergence among K weak-model samples with no external verifier, and exposes a **compression ratio that calibrates answer reliability**; and (ii) a **Certified-Depth Accumulator**, in which the weak model drills sub-problems, *certifies each by execution*, banks the verified building blocks, and *composes* them on harder problems.

Three findings. (1) On a 5-task K-scaling suite, weak+MDL at K=20 reaches mean 0.843 vs strong-at-K=1 0.700, and matches or beats the strong model on 5/5 tasks; on the hardest grid task the weak model goes 0.29→0.86→1.00 as K grows while the strong model stays at 0.00. (2) The compression ratio is a usable uncertainty certificate: when ratio ≥ 2.0 , accuracy is 0.90 (8/10 perfect). (3) The accumulator creates capability that *neither cold weak nor cold strong* has: on held-out composite rules, cold gpt-4o-mini and cold gpt-4o both solve 0.00 (all three seeds), while the *same* gpt-4o-mini plus a self-drilled, execution-certified library reaches 0.25–0.75 — capability appearing only once verified vocabulary is banked. The signal is clean at the mechanism level (cold = 0 vs library > 0) but noisy in magnitude (coarse 4-task solve rate); we report it honestly as such.

We are explicit about the boundary: the mechanism's strength is bounded by the strength of the available verifier. On a real verifier-poor benchmark (DiscoveryBench-synth) weak+MDL still matches strong on 9/12 tasks, but accuracy saturates near 0.5 and the calibration signal goes flat — exactly because there is no execution oracle. Full official benchmark evaluations against published SOTA are the next step, not a claim of this draft.

1. Introduction

The gap we attack. Inference-time methods (self-consistency, best-of-N) only *resample a fixed prior*: if the answer is not already in the weak model's distribution, sampling it 20 times does not help. A closed loop with the model as its own judge is worse — it reinforces its own errors. Genuine improvement of a frozen model therefore requires an *external source of truth-pressure that is not a stronger model*. The cheapest universal one is **execution**.

Thesis. A frozen weak model cannot increase its per-step computation, but it *can* increase its **reachable depth** by banking results it has verified by execution. A theorem at search-depth 40 today becomes depth-0 (a lookup) tomorrow once it is derived and certified. This is the difference between a first-year student and a 30-year professor: not faster neurons, but an organized, *verified* store of results to compose. We call the artifact a *certified library* and the process *certified-depth accumulation* (informally, "drilling" — ■■■■■■■■).

Why bits are the judge. We frame every task as a compression problem (Solomonoff/MDL, Rissanen): the best explanation of observations is the shortest program that regenerates them, $L(\text{program}) + L(\text{data} \mid \text{program})$. Crucially, $L(\text{data} \mid \text{program})$ is measured by *running the program*. This makes Occam's razor intrinsic (a hallucinated complex answer pays more bits) and makes the criterion task-agnostic: the same engine drives sequence rules, grid effects, symbolic laws, and pandas analyses.

Contributions. 1. **Variation-MDL**: a verifier-free amplifier that compresses the *variation* among K weak-model responses into an invariant core, and yields a **calibration signal** (compression ratio) predicting reliability. 2. **Certified-Depth Accumulator**: a frozen-weights, in-context "drill → certify-by-execution → bank → compose" loop, with an MDL gate so only building blocks that reduce future description length are kept. 3. **DPSR — Bayesian typed-hole hypothesis induction**: a universal pre-proposal layer that changes the unit of generation from complete hypotheses to typed holes inferred by information-gain micro-measurements; and its composition with the accumulator (structure from the certified library, coefficients by measurement), which lifts held-out composite solve from 0.341 to 0.863. 4. **An honest map of where weak $\geq$ strong holds**: the dividing line is verifier strength, demonstrated on both verifier-rich (synthetic, clean execution) and verifier-poor (real DiscoveryBench) settings.

2. Method

2.1 The MDL engine (solving = shortest certified program)

A task is a `CompressionTask`: a stream of observations plus a way to *run* a candidate program on one observation. The engine proposes K candidate programs (from a weak LLM), compiles them in a sandbox, scores each by `program_bits + residual_bits` where residual is charged per observation the program fails to reproduce *under execution*, and keeps the candidate with the best fit (ties broken by fewer bits). Accepted programs with exact fit are promoted to a **Library**; library calls are cheap in bits, so later tasks are encouraged to reuse

them — this is where transfer lives.

2.2 Variation-MDL (amplification without a verifier)

When no executable verifier exists, the K responses *are* the observations. We run the weak model K times into a structured schema (mechanism / formula / threshold / direction / answer), then select the **MDL-optimal core**: among candidate cores (the modal answer, each response, and a meta-synthesised one) we pick $\text{argmin} [L(\text{core}) + \sum L(R_i \mid \text{core})]$, with field-level residuals (log-error for numbers, normalized edit distance for strings). The ratio $\text{baseline_bits} / \text{mdl_bits}$ measures how much real structure was found; we show below it predicts when the answer can be trusted. Probes are then aimed at the highest-entropy residual fields (max information gain) for the next round.

2.3 Certified-Depth Accumulator (drill → certify → bank → compose)

The amplifier above cannot create knowledge that is wholly absent. The accumulator can grow *reachable* knowledge:

1. **Drill.** The weak model practices a sub-problem on freshly sampled data, retrying until it produces a program that **reproduces all observations under execution** (exact rate ≥ 0.99). No proof, no bank — certification is the price of entry, and it is what prevents the loop from banking hallucinations.
2. **Bank.** The certified solver is renamed to a unique callable ($f \rightarrow \text{prim_<name>}$) and added to the library as a reusable building block.
3. **Compose.** On a harder task, the library is supplied; the model is nudged that the answer is most likely a *composition* of known blocks ($\text{return prim_a(ctx) + prim_b(ctx)}$). It must now discover only the combination, not re-derive each piece — turning a $p \approx 0$ search into a feasible one.

The MDL gate ensures the library stores generalisations, not memorised answers, and held-out evaluation snapshots the library so a solved test task cannot leak back into it.

2.4 DPSR — Bayesian typed-hole hypothesis induction

The methods above SELECT and VERIFY hypotheses; they do not change the unit of generation. A weak model asked for a whole hypothesis fails when $p(\text{hypothesis}) = \prod p(\text{part}) \approx 0$. DPSR changes the unit of generation:

```
not: weak LLM -> a complete hypothesis but: weak LLM -> a hypothesis SKELETON with typed holes  $\theta$  engine -> infers each hole's value by executable micro-measurement engine -> recombine + verify; on failure, blame and reopen holes
```

A skeleton is a program with its unknowns exposed as a top-level dict $H(\theta = \{\theta_1 \dots \theta_n\})$. For each hole the engine maintains a posterior $p(\theta_i \mid D)$ whose likelihood for a candidate value is $\exp(-\text{loss}/\tau)$ measured by EXECUTING the instantiated program on observations (via the adapter). Holes are closed in order of uncertainty $\times$ downstream impact; structural holes are measured WITH a greedy refit of dependent holes (a Bayesian dependency graph), and free constants are calibrated per candidate so values that differ only by a constant remain distinguishable. A closed skeleton is recombined and verified; on residual, $\text{blame} \propto$ each hole's contribution to the residual reopens only the guilty hole.

The weak model only writes (i) the skeleton shape and (ii) tiny per-hole measurement code; every VALUE is inferred from data, never guessed. DPSR is a universal pre-proposal layer in the engine: it drives only the adapter contract (`signature_hint` / `interface_description` / `sandbox_globals` / `execute` / `loss` / `calibrate`), so the same code runs on every benchmark with no task-specific logic.

CDA $\times$ DPSR. The two new methods compose: the certified building blocks banked by the accumulator become the blocks of a DPSR skeleton $f(x) = \sum H[c_i] \cdot \text{block}_i(x)$, and DPSR infers the coefficients (subset selection) by measurement. Structure comes from the verified library; which blocks and weights are inferred from data.

3. Experiments

All experiments use weak = gpt-4o-mini, strong = gpt-4o. Code: `mars/mdl/`, `mars/runners/`. Result files: `lmw/kscale/`, `lmw/accumulator/`.

3.1 K-scaling: weak + MDL $\geq$ strong (run `kscale_v2`)

Five tasks (one execution-aligned grid task, four hypothesis tasks). weak+MDL is evaluated at $K \in \{1,3,5,10,20\}$; strong is evaluated once ($K=1$).

metric	value
weak+MDL (K=20), mean	0.843
strong (K=1), mean	0.700

tasks where weak+MDL_best $\geq$ strong	5 / 5
---	-------

Hardest case (execution-aligned grid letter E), weak+MDL solve rate vs K:

K=1: 0.29 $\rightarrow$ K=3: 0.86 $\rightarrow$ K=5: 1.00 (strong K=1: 0.00)
--

The weak model, given compute to vary and a bit-based selector, climbs from near-zero to perfect, while the single strong sample never finds the structure.

3.2 Calibration: compression ratio predicts reliability (run ksca1e_v2)

Pooling all (compression_ratio, accuracy) points:

threshold	n	mean accuracy	perfect
ratio $\geq$ 1.5	16	0.75	10/16
ratio $\geq$ 2.0	10	0.90	8/10
ratio $\geq$ 2.3	7	0.86	5/7

The compression ratio behaves as an **uncertainty certificate**: high ratio $\rightarrow$ the engine has found stable structure $\rightarrow$ the answer can be trusted, *without an oracle*. This is, to our knowledge, the most directly useful by-product of the method.

3.3 Certified-Depth Accumulator: accumulation creates capability (run acc_v2)

Held-out composite tasks are sums of 2–4 primitive grid effects (parity, threshold, modulo, visit, corner, diagonal). The weak model drills the six primitives one at a time (each certified by execution before banking) and the held-out solve rate is re-measured as the vocabulary grows. Test evaluation uses a library snapshot, so no test answer can leak into the library.

Solve rate (fraction of 4 held-out composites fully cracked), averaged over 3 seeds {7, 38, 78}:

condition	solve rate (avg of 3 seeds)	per-seed
cold gpt-4o-mini (empty library, asked directly)	0.00	[0.0, 0.0, 0.0]
cold gpt-4o (empty library, raw strong model)	0.00	[0.0, 0.0, 0.0]
gpt-4o-mini + self-drilled certified library	0.25	[0.25, 0.0, 0.5]

Compounding curve (single seed, solve rate vs number of banked certified primitives):

banked 0: 0.00 banked 2: 0.25 (t_pt cracks once parity+threshold are banked) banked 3: 0.50 banked 4: 0.75 (rises in lock-step with the banked vocabulary) banked 5: 0.00 (noise: solve rate over 4 tasks is coarse, step = 0.25) banked 6: 0.25 (full library not reliably better than 4 – see below)
--

Two honest readings. **(i) The robust claim:** both cold baselines — including the strong model — score **0.00 on all three seeds**, while gpt-4o-mini with a self-drilled certified library reaches 0.25–0.75. Capability appears *only* once verified vocabulary is banked; it is created by accumulation, not by model scale. The rising part of the curve (0 $\rightarrow$ 0.75 as the first four primitives are banked) shows capability tracking the vocabulary. **(ii) The honest caveat:** the result is *noisy*. Solve rate over only 4 composites is coarse (steps of 0.25), and the full 6-primitive library is **not reliably better** than 4 — with more building blocks in context the model has more ways to compose the wrong combination, so selection gets harder. A stable headline needs more test composites and more seeds; this is a clean *mechanism* result, not yet a polished benchmark number.

3.4 The honest boundary: verifier-poor caps out (run db_ksca1e_v1)

On 12 real DiscoveryBench-synthetic tasks (column descriptions + sample rows + correlations; no code execution), weak+MDL still matches or beats strong on **9 / 12** tasks. But:

- accuracy saturates near **0.5** (the model gets the *direction* right — positive/negative relation — but not the exact threshold/variable), and
- the calibration signal goes **flat**: at ratio $\geq$ 2.0, mean accuracy is only 0.44 (0/34 perfect), versus 0.90 in the verifier-rich setting.

This is the expected and important negative result: **the mechanism's power is bounded by the verifier**. Where there is no execution oracle, compression of text alone cannot certify the missing detail, and the compression ratio stops predicting correctness.

3.5 CDA × DPSR: structure from the library, values by measurement (run dpsr_cda_v1)

On held-out composite rules (sums of 2–4 grid primitives), DPSR with the accumulator's certified library is compared to DPSR that must invent the additive structure itself (cold). Held-out solve rate (1.0 = fully cracked):

composite	cold DPSR	DPSR + certified library
parity + threshold	0.36	1.00
parity + threshold + modulo	0.27	1.00
corner + diag + visit	0.55	0.73
parity + threshold + modulo + visit	0.18	0.73
mean	0.341	0.863

The crossing lifts mean solve from 0.341 to **0.863** (2.5×). Where every needed primitive is in the library, DPSR fully cracks the composite by measuring which coefficients to switch on. The two partial cases are partial for an *honest, explainable* reason: the *visit* primitive failed certification (exact rate 0.96, below the 0.99 gate) and so was never banked — both partial composites require it, and DPSR correctly assembled only the available subset. The certification gate refusing an unproven block, with a visible and attributable consequence, is the mechanism behaving as designed.

3.6 DPSR is universal: same engine, a second domain (run dpsr_nb_v3)

To show DPSR is not specialised to grid rules, the *same* engine — through the *same* adapter contract, no new code — induces numeric scientific laws. The law's multiplicative constant is fit by the adapter's *calibrate*; DPSR infers the exponents/structure by measurement. Held-out relative error vs raw one-shot weak:

law	DPSR rel-err	raw weak rel-err
gravity $F = G \cdot m \blacksquare \cdot m \blacksquare / r^2$	0.000	0.000
pendulum $T = 2\pi\sqrt{(L/g)}$	0.081	0.472
kinetic $KE = \frac{1}{2}mv^2$	0.442	0.000

DPSR beats raw weak on **2/3** laws and is exact on gravity — a domain with different structure (exponents, a constant spanning many orders of magnitude), reached with no benchmark-specific code. This confirms the **universality of the mechanism**. The honest limit: on *memorised textbook* laws (kinetic energy) raw recall is already exact and DPSR's decompose-then-measure can underperform — DPSR's value is concentrated in the **discovery** regime where one-shot recall fails. (NewtonBench's own counterfactual law shifts are designed precisely to defeat recall, i.e. the regime where DPSR should help most; an official run is future work.)

4. When it works and when it does not

regime	example	verifier	expected result
verifier-rich	code, symbolic-law induction, our composite tasks	execution / unit tests	weak + accumulation can match or beat strong
verifier-poor	DiscoveryBench HMS, open scientific interpretation	noisy / none	direction recoverable; exact claim caps out

The single design variable that decides the outcome is **the strength of the executable check**. A useful consequence: the compression ratio *tells us in advance* which regime a task is in.

5. Related work

- **Self-consistency / best-of-N** (Wang et al. 2022; AlphaCode): resample a fixed prior. We add a bit-based selector and, crucially, a calibration signal; and the accumulator adds *new certified content*, which resampling cannot.
- **Program-verified reasoning** (AlphaProof 2024; DeepSeek-Prover): weak policy
- perfect verifier + search reaches high. We keep the verifier idea but stay *weight-frozen* and seek cross-domain generality rather than RL on one domain.

- **Self-improvement** (STaR; ReST; Absolute Zero Reasoner 2025): closest in spirit; AZR proposes-and-solves with code execution and **RL fine-tuning**. Our distinction is the **frozen-weights, in-context** library curated by an **MDL gate** (keep only what compresses), turning the API constraint (can't train) into the thesis (an external verified memory replaces training).
- **Skill libraries** (Voyager 2023): we frame the library as *reachable-depth banking* with an explicit compression/certification criterion for admission.
- **Foundations**: MDL/Kolmogorov (Solomonoff; Rissanen) for the objective; von Neumann (1956) reliable computation from unreliable components and Bennett (1988) logical depth for *why* accumulation, not per-step power, is the lever for a frozen model.

6. Limitations (stated plainly)

1. **No full official-benchmark SOTA comparison yet.** Our strong numbers are on synthetic verifier-rich tasks and on slices; the real DiscoveryBench number is below SOTA by design (no execution). Official evals (Docker ready) are the next step.
2. **Composite domain is synthetic.** It cleanly isolates the mechanism but is not a public benchmark; transfer to real verifier-rich benchmarks (HumanEval/MBPP, NewtonBench) is unproven here.
3. **Coverage limit.** Accumulation requires the weak model to propose each building block with non-zero probability. Where a needed primitive is wholly outside its repertoire, no banking helps — the honest residue of the "if it doesn't know it, it can't solve it" objection.
4. **Single-judge / single-seed noise** in some sub-experiments; we average over seeds for the headline accumulator verdict but more trials are warranted.

7. Next steps

1. Scale the accumulator from synthetic composites to **one verifier-rich public benchmark** end-to-end (code or NewtonBench), and report against published SOTA — the first true SOTA-relative number.
2. Run the **full official evaluations** (Docker) for the four benchmarks once the universal engine is fixed, replacing all demo placeholders with measured numbers.
3. Strengthen verifier-poor performance with **synthesised checks** (reduce an unverifiable claim to executable sub-claims), the only honest route past the 0.5 ceiling.

```
Reproduce: python -m mars.runners.run_k_scaling --run_id kscale_v2 --overwrite · python -m
mars.runners.run_db_kscale --run_id db_kscale_v1 · python -m mars.runners.run_accumulator --run_id acc_v2
--overwrite
```